Deep Dive AI

Workshop Series

SESSION 01

Foundations of Large Language Models

Architecture, training pipelines, and capabilities of modern LLMs

Session 1 · March 2026

Aryan Mishra

Agenda

01 **Foundations of LLMs**
What LLMs are, how they learn from large-scale text data, and the role of neural networks.

02 **Core Architecture**
Transformer architecture and how LLMs generate text by predicting the next token.

03 **Model Training & Improvement**
Fine-tuning techniques and reinforcement learning with human feedback (RLHF).

04 **Applications & Capabilities**
Common real-world uses of LLMs across various domains.

05 **Limitations & Reasoning**
Cognitive limits of LLMs and how reasoning methods and external tools help mitigate them.

06 **Challenges & Risks**
Technical limitations, security challenges, and ethical considerations.

07 **Emerging AI Landscape**
Multimodal AI, autonomous agents, and voice-based systems.

08 **Current AI Landscape (2026)**
Overview of major models and the current AI ecosystem.

09 **Hands-on Development**
Building an LLM wrapper from scratch.

Foundations of LLMs

FOUNDATIONS

01 What is a Language Model?

LANGUAGE MODEL

A system designed to predict the next word, phrase, or character in a sequence based on preceding context. Uses patterns learned from vast text data to produce coherent, relevant outputs.

BEST FOR

Autocomplete at scale — text prediction from context

USE WHEN

Introducing LLMs from first principles

02 When Does It Become "Large"?

SCALE THRESHOLD

When trained on extremely large datasets with billions of parameters. This scale enables broader context understanding, complex pattern capture, and emerging capabilities like reasoning and summarisation.

BEST FOR

Billions of params — emergent reasoning emerges

USE WHEN

Explaining scale effects and capability jumps

03 How Does an LLM Look Internally?

INTERNAL ARCHITECTURE

Two key components: a Parameter File (trained weights storing learned knowledge) and Runtime Code (software that loads weights and performs inference to generate outputs from input prompts).

BEST FOR

Parameter File + Runtime Code = LLM system

USE WHEN

Building intuition before diving into Transformers

Foundations of LLMs

FOUNDATIONS — the AI hierarchy, NLP scope, and where LLMs sit

The Hierarchy

Artificial Intelligence (AI)

Broad domain — machines performing intelligent tasks

Natural Language Processing (NLP)

Subfield — understanding and generating human language

Language Models (LMs)

Models that predict or generate text sequences

Large Language Models (LLMs)

Very large models trained on massive datasets via Transformers

What NLP Includes

Large Language Models are a specialised subset within a broader AI hierarchy, focused on understanding and generating human language at scale.

- Text classification
- Sentiment analysis
- Named Entity Recognition (NER)
- Machine translation
- Speech-to-text
- Question answering
- Chatbots

These tasks can use rule-based, statistical, or deep learning (LLM) approaches.

How an LLM Processes Your Input

FOUNDATIONS

① **Prompt**

Raw user text input — e.g. "What is an LLM?"

↓

② **Encode (Tokenizer)**

Text normalisation → subword segmentation → token IDs: [1294, 338, 385, 9231, 29973]

↓

③ **Inference Engine**

Python / C++ runtime: load weights, run transformer layers, compute next-token logits

↓

④ **Decode (Tokenizer)**

Token IDs back to text: [1294, 338…] → "An LLM is…"

↓

⑤ **Response**

Final text displayed to user: "An LLM is a large language model…"

MODEL WEIGHTS

70B params (~140 GB)

- Attention weights
- Feed-forward weights
- Embedding matrices
- Layer normalisation

NEXT TOKEN PREDICTION

The inference engine produces a probability distribution over the vocabulary. The highest-probability token is sampled. This loop repeats until generation is complete.

Parameter file is static at runtime — only the run file executes. Inference is purely mathematical.

02

How Are LLMs Built?

Data collection, pre-training, fine-tuning, and RLHF — the full lifecycle of a modern LLM

Pre-training | Fine-tuning | RLHF

Pre-Training

PRE-TRAINING — Building the Foundation of an LLM

01 Data Acquisition

DATA COLLECTION

Collecting large volumes of text from diverse sources: websites, books, articles, forums, and public datasets to form the raw training corpus.

BEST FOR

Diverse sources → broad model knowledge

USE WHEN

Introducing the first step of LLM pre-training

02 Data Filtering & QC

QUALITY ASSURANCE

Cleaning and refining collected data to remove noise, low-quality text, duplicates, and sensitive information before the model sees it.

BEST FOR

Clean data → safe, effective training corpus

USE WHEN

Explaining why raw web data can't be used directly

03 Tokenization

TEXT PREPROCESSING

Converting raw text into smaller units (tokens) the model can process. Maps human language to numeric representations for the neural network.

BEST FOR

Text → token IDs → model can process

USE WHEN

Setting up the tokenization deep-dive in the next section

The quality of pre-training data directly determines the quality of the model's knowledge and reasoning ability.

Data Acquisition

PRE-TRAINING — Collecting Large-Scale Training Data

THE ACQUISITION FLOW

LLMs are trained on massive datasets gathered from the internet and other publicly available sources — processed through a three-stage acquisition pipeline.

The Internet

Vast amounts of text across websites, blogs, research papers, forums, and documentation.

↓

Web Crawlers

Automated tools that systematically visit billions of web pages and extract textual content.

↓

Training Dataset

Aggregated text compiled into large corpora — the raw training data for the LLM.

REAL-WORLD EXAMPLE

EXAMPLE DATASET

FineWeb Dataset

Available on Hugging Face

A curated large-scale dataset derived from web content, commonly used for training modern language models. It demonstrates what a real-world training corpus looks like after data acquisition.

Results in a massive raw dataset — hundreds of billions of tokens — requiring significant cleaning before training.

SCALE CONTEXT

Modern LLM training datasets contain trillions of tokens. GPT-4 is estimated to have been trained on over 13 trillion tokens from diverse internet and book sources.

Data Filtering and Quality Control

PRE-TRAINING — Cleaning and Preparing the Dataset

1

URL & Domain Filtering

Removing low-quality or unreliable sources before any text is even extracted.

2

Text Extraction Filtering

Retaining only meaningful textual content; stripping HTML, scripts, and boilerplate.

3

Language Detection Filtering

Keeping only the target language(s) and discarding non-relevant content.

4

Duplicate Removal

Eliminating repeated content to improve dataset diversity and prevent model memorisation.

5

Privacy & Sensitive Content

Removing personal data, harmful content, and sensitive information before training begins.

Result: A clean, high-quality dataset that can be safely and effectively used for training the language model.

Tokenization

TOKENIZATION — From Characters to Byte Values

STEP 1 → 1D Text Sequence

Raw text as a flat character stream

```
in the early years of computing scientists
began exploring whether machines might
one day process human language...
```

STEP 2 → UTF-8 Binary Encoding

Each character encoded as binary bits

```
01101001 01101110 00100000 01110100
01101000 01100101 00100000 01100101
01100001 01110010 01101100 01111001
```

STEP 3 → Byte Values

Binary patterns read as integers

```
[105, 110,  32, 116, 104, 101,  32,
 101,  97, 114, 108, 121, ...]
```

CHAR → UTF-8 BINARY → BYTE

CHAR	UTF-8 BINARY	BYTE
i	01101001	105
n	01101110	110
(sp)	00100000	32
t	01110100	116
h	01101000	104
e	01100101	101
(sp)	00100000	32
e	01100101	101
a	01100001	97
r	01110010	114

Each character maps to a unique UTF-8 binary pattern, read as an integer byte value by the model.

Byte Pair Encoding (BPE)

TOKENIZATION — Iterative Subword Merging

BUILDING THE VOCABULARY

STEP 1 • Starting Text

```
in the early years
```

Fragment of raw training text

STEP 2 • Initial Symbols

```
i n _ t h e _ e a r l y _ y e a r s
```

_ = space. Split into individual characters

STEP 3 • Count Frequent Pairs

```
(t,h)  (h,e)  (e,r)  (e,a)  (l,y)
→ Most frequent: th, he, er, ea, ly
```

Pairs counted across the full dataset

MERGE OPERATIONS

STEP 4 • First Merge: th

```
t h → th
i n _ th e _ e a r l y _ y e a r s
```

Most frequent pair merged first

STEP 5 • Second Merge: the

```
th e → the
i n _ the _ e a r l y _ y e a r s
```

Continue with next frequent pair

STEP 6 • Final Tokens + IDs

```
in | the | ear | ly | years
→  [452, 12, 981, 77, 2045]
```

Each token maps to a numeric ID

Try it live: tiktokenizer.vercel.app

Full Tokenization Pipeline

TOKENIZATION — End-to-End Text to Token IDs

THE PIPELINE

How raw text travels through the tokenization process before reaching the transformer.

Raw Text	Natural language input
UTF-8 Bytes	Each character → binary → byte integer
Initial Symbols	Characters / bytes split into base units
BPE Merge Ops	Frequent pairs iteratively merged
Subword Tokens	in | the | ear | ly | years
Token IDs	[452, 12, 981, 77, 2045]

Token IDs → embedding vectors → transformer layers

EXAMPLE WALKTHROUGH

Raw Text

"in the early years"

UTF-8 Bytes

[105, 110, 32, 116, 104, 101, 32...]

Characters

i n _ t h e _ e a r l y

After BPE

in | the | ear | ly | years

Token IDs

[452, 12, 981, 77, 2045]

Next Step

IDs → embeddings → transformer

Neural Network Training

FOUNDATIONS — Transforming Random Parameters into Language

THE CONCEPTS

Modern LLMs rely on two core technologies: neural network training and the transformer architecture.

1 Billions of Random Parameters

At the start of training the model contains billions of adjustable values initialised randomly — no knowledge of language whatsoever.

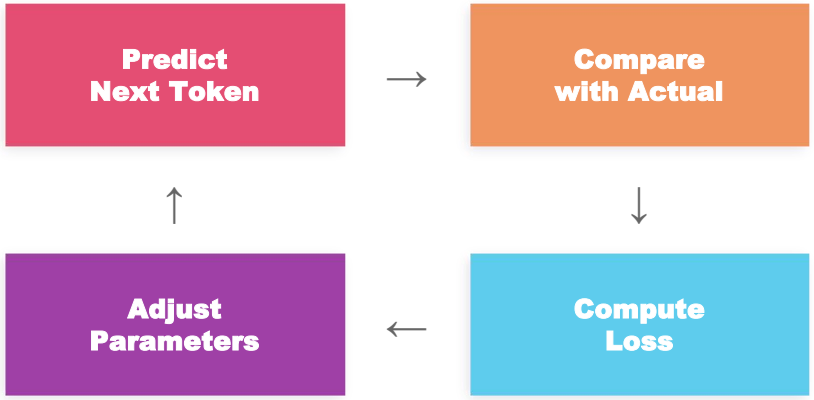
2 Next-Token Prediction Loop

The model predicts the next token, compares with the actual token using a loss function, and adjusts parameters to reduce error — repeated billions of times.

3 Predictive Typing Analogy

Like predictive typing on a smartphone — early predictions seem random, but the system steadily improves as it learns patterns from data.

THE TRAINING LOOP



Repeated billions of times per training run

Each update uses gradient descent — adjusting weights by tiny amounts in the direction that reduces prediction error.

Emergent capabilities arise after trillions of updates — the model was never explicitly taught them.

Scaling Large Language Models

SCALING — Why Size Changes Everything

175B **parameters in GPT-3**

As models grow larger, their capabilities improve significantly — but so do their training requirements.

01 High-Performance GPUs

AI ACCELERATORS

AI accelerators such as NVIDIA H100, capable of massive parallel mathematical computation required for training.

Hundreds of H100s per training run

02 Massive Compute

SCALE OF OPERATIONS

Extremely large numbers of mathematical operations (FLOPs) performed across enormous datasets over weeks or months.

Billions of operations per second

03 Distributed Training

INFRASTRUCTURE

Workloads split across hundreds or thousands of machines to handle the computational demand at scale.

Across data centres globally

Scale is what differentiates modern LLMs. Massive parameters + enormous datasets + distributed GPU infrastructure = emergent reasoning, summarisation, and conversation capabilities.

From Training to Real-World Use

INFERENCE — The Transition from Learning to Applying

THE TRANSITION

Once training is complete, the model transitions from learning patterns to applying them in real-world tasks. This stage is known as inference.

Training

Learning patterns from data

↓

Inference

Applying knowledge to real-world tasks

WHAT IS INFERENCE?

The stage where the model uses its learned knowledge to perform tasks — this is when it becomes a practical tool for end users.

INFERENCE APPLICATIONS

01 Text Generation

Creating coherent, context-aware text from user prompts

02 Summarisation

Condensing long content into key points and takeaways

03 Translation

Converting text accurately between human languages

04 Question Answering

Providing precise answers derived from context or knowledge

05 Content Creation

Generating creative, professional, and structured content

How LLM Inference Works

INFERENCE — From Prompt to Generated Response

THE INFERENCE PIPELINE

Inference is the process by which a trained LLM generates text from a user prompt.

01 — Tokenization

Input text is broken into tokens using the trained vocabulary (e.g. BPE)

02 — Token IDs

Each token converted into its corresponding numerical ID from the vocabulary

03 — Next-Token Prediction

Model evaluates all vocabulary tokens and assigns probability scores to each

04 — Sequence Generation

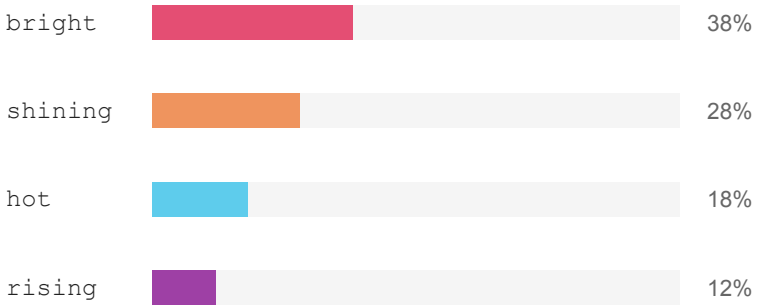
Tokens generated one at a time, each appended to the sequence and fed back in

WORKED EXAMPLE

PROMPT

```
"The sun is"
```

POSSIBLE NEXT TOKENS



The model assigns probability scores to every token in its vocabulary, then samples or selects based on the score distribution.

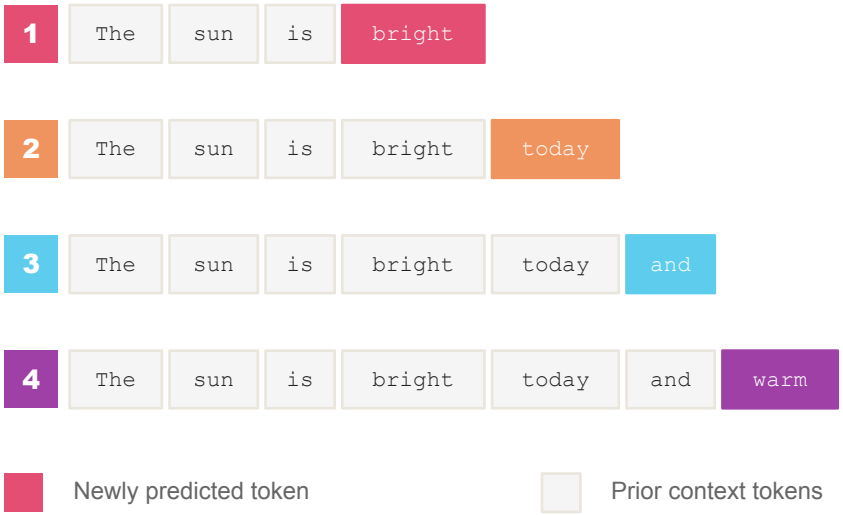
The selected token is appended to the sequence and the process repeats until a stop condition is met.

Autoregressive Text Generation

INFERENCE — One Token at a Time

STEP-BY-STEP GENERATION

LLMs generate text using an autoregressive process — one token at a time, each depending on all previous tokens.



HOW IT WORKS

Token 1 context:
`prompt only`

Token 2 context:
`prompt + token 1`

Token 3 context:
`prompt + tokens 1–2`

Token N context:
`all previous tokens`

CONTEXT WINDOW

Each token prediction depends on the entire sequence generated so far, allowing the model to maintain contextual coherence throughout the response.

"Autoregressive" means each output token becomes input for the next prediction — text is built sequentially, never in parallel.

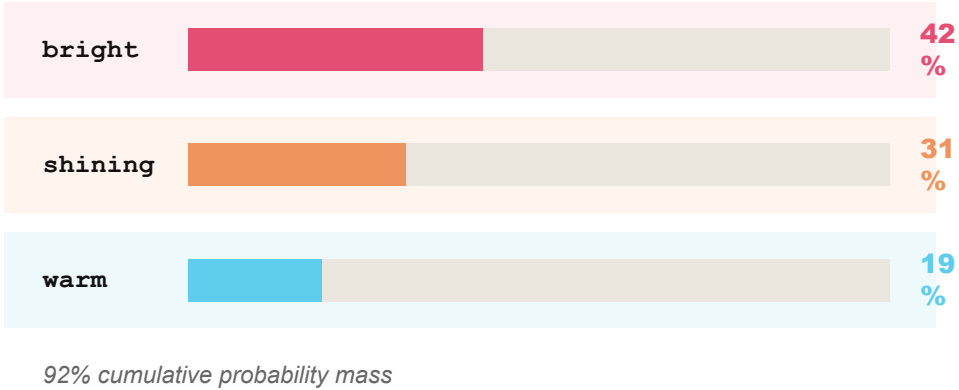
Nucleus Sampling & Temperature

INFERENCE — Controlling Randomness in Generation

TOP-P (NUCLEUS) SAMPLING

Instead of a fixed number of tokens, the model selects from the smallest set whose cumulative probability exceeds a threshold.

CANDIDATE SET (cumulative p > 90%)



92% cumulative probability mass

A token is randomly sampled from this nucleus — maintaining diversity while excluding very unlikely outputs.

TEMPERATURE CONTROL

Temperature adjusts the randomness of token selection during generation — from deterministic to highly creative.

0.0 Focused — Creative 2.0

Low Temperature

Deterministic, precise output

```
"The sun is bright."
```

High Temperature

Creative, diverse output

```
"The sun is blazing across the sky."
```

Most production systems use temperature 0.7–1.0 to balance coherence with variability.

Supervised Fine-Tuning (SFT)

FINE-TUNING — From Next-Word Predictor to Conversational Assistant

THE TRANSFORMATION

During pre-training, the model learns to predict the next word based on statistical patterns. While this enables strong language understanding, it does not automatically produce structured human conversations.

Pretrained Model

Next-word predictor

↓ *Supervised Fine-Tuning*

Conversational Assistant

Helpful, truthful, and harmless

SFT TRAINS THE MODEL TO PRODUCE RESPONSES THAT ARE:

Helpful | **Truthful** | **Harmless**

EXAMPLE

TRAINING CONVERSATION

User:

"Can you recommend landmarks in Paris?"

Assistant:

"You could visit the Eiffel Tower, the Louvre Museum, or Notre-Dame Cathedral."

Goal: Clear, informative, conversational

SFT shifts the model's objective from next-word prediction to producing responses that actually help people.

The SFT dataset is curated by human annotators following strict quality guidelines.

Creating the Conversation Dataset

FINE-TUNING — Human-Annotated Training Data

DATASET COMPONENTS

SFT relies on high-quality conversational datasets crafted by human annotators following clear guidelines.

01 **User Questions**

Natural language prompts representing real-world user queries — broad and diverse.

02 **Ideal Assistant Responses**

Carefully crafted answers that are helpful, accurate, and safe — written by trained annotators.

ANNOTATOR GUIDELINES

Helpful | **Accurate** | **Safe**

Data quality is the limiting factor for SFT — a small set of excellent conversations outperforms a large set of mediocre ones.

CONVERSATION STRUCTURE

User: `Question`

Assistant: `Response`

This structure is repeated across the dataset.

WHAT THE MODEL LEARNS

1. **Dialogue Flow**
   How conversations naturally progress and transition between topics
2. **Multi-Turn Conversations**
   Handling sequences of exchanges while maintaining coherence
3. **Context Awareness**
   Maintaining and referencing prior context across multiple turns

Training the Model with SFT

FINE-TUNING — Updating Parameters via Backpropagation

THE TRANSFORMATION

During SFT, the model's parameters are updated so it learns to generate responses similar to curated human examples.

Before SFT
Statistical Next-Word Prediction

↓ *Supervised Fine-Tuning*

After SFT
Human-Aligned Conversation

TRAINING VIA BACKPROPAGATION

Billions of parameters are gradually adjusted to reduce the gap between the model's generated responses and the ideal human-written examples.

WHAT SFT IMPROVES

01 Natural Language Flow
Smooth, human-like sentence construction and natural transitions

02 Context Awareness
Understanding and responding to the full conversation history

03 Response Accuracy
Generating factually correct and contextually relevant answers

RESPONSE COMPARISON

Before: *"Paris is a city in France that has many historical..."*

After: *"Sure! Here are some great spots to visit in Paris..."*

The model starts behaving like a conversational assistant rather than a pure text predictor.

Personality & Multi-Turn Dialogue

FINE-TUNING — Consistent Style and Extended Conversations

CONVERSATIONAL TONES

Depending on training data, models can adopt different conversational styles for different contexts and audiences.

Friendly & Approachable

"Hey! Great question — let me help you with that!"

Professional & Formal

"The recommended approach is as follows."

Informative & Instructional

"Step 1: Open the terminal. Step 2: Run the command."

MULTI-TURN CAPABILITIES

Training with multi-turn dialogue examples helps the model learn to handle extended interactions naturally.

01 **Reference Earlier Messages**
Connecting current responses to previous exchanges in the thread

02 **Maintain Context**
Keeping track of topics, entities, and user intent across turns

03 **Coherent Over Multiple Turns**
Avoiding contradictions and staying on topic throughout a session

Interactions feel more natural, engaging, and contextually aware — the model remembers what was said.

REAL-WORLD IMPACT

Multi-turn training is what separates one-shot Q&A bots from genuinely conversational AI assistants.

Reinforcement Learning with Human Feedback

RLHF — Aligning Responses with Human Preferences

THE RLHF WORKFLOW

Unlike SFT which teaches by examples, RLHF teaches by ranking responses — training the model to prefer what humans prefer.

01 — Generate Candidates

Model produces multiple candidate responses for a given prompt

02 — Human Ranking

Human annotators compare and rank responses from best to worst

03 — Train Reward Model

A reward model learns these human preferences as a scoring function

04 — Optimize the Model

Model is optimized to generate responses that receive higher rewards

RLHF IMPROVES

Response Relevance

Answers are more directly useful, on-topic, and actionable for the user

Tone & Politeness

Appropriate, respectful communication style adapted to the conversation

Safety & Alignment

Responses better aligned with user expectations, guidelines, and values

RLHF is what makes models helpful, harmless, and honest — the foundation of modern AI alignment.

USED BY

GPT-4, Claude, Gemini, and most major LLMs use RLHF or variants as part of their alignment training pipeline.

Challenges & New Approaches

RLHF — Limitations of RLHF and the Rise of DPO

RLHF CHALLENGES

While RLHF significantly improves model behaviour, it introduces several challenges that have driven the development of alternatives.

Reward Hacking

!

The model may learn to exploit the reward function — achieving high scores without genuinely improving response quality.

High Cost

$$

Collecting human feedback at scale is expensive and time-intensive — each ranking requires human annotation effort.

Balancing Creativity & Accuracy

Difficult to simultaneously optimise for both creative responses and factual accuracy without quality trade-offs.

DIRECT PREFERENCE OPTIMIZATION

A Newer Alternative

DPO simplifies the alignment training process by eliminating the need for a separate reward model entirely.

- ✓ Directly optimises the model using human preference comparisons
- ✓ Avoids training and maintaining a separate reward model
- ✓ Reduces training complexity and overall compute cost

Many modern systems combine SFT, RLHF, and DPO to produce high-quality conversational AI.

KEY TAKEAWAY

No single method dominates — production systems combine techniques to maximise safety, quality, and efficiency.

Applications & Capabilities

APPLICATIONS — Common Real-World Use Cases Across Domains

Large language models power practical systems across industries by enabling natural language understanding, generation, and reasoning.

01

Conversational Agents

Virtual assistants, chatbots, and multi-turn dialogue systems for customer support and enterprise workflows.

02

Content Creation

Drafting articles, summarisation, translation, grammar correction, and marketing copy generation at scale.

03

Coding Support

Code generation, autocompletion, debugging, refactoring, and test case creation — significantly improving developer productivity.

04

Knowledge & QA

Question answering, structured explanations, research assistance, and intelligent knowledge retrieval from large corpora.

These capabilities make LLMs foundational infrastructure for intelligent software systems across every industry.

Conversational Agents & Content Generation

APPLICATIONS — Use Cases in Communication and Writing

Conversational Systems

1. Chatbots for customer support
2. AI assistants for enterprise workflows
3. Multi-turn contextual dialogue systems
4. Automated service agents in applications

Maintain context across interactions and generate human-like responses.

Text Summarisation

1. Condensing long reports into short insights
2. Extracting key points from research papers
3. Creating executive summaries from documents

Content & Writing Assistance

1. Drafting blogs, articles, and technical documentation
2. Grammar correction and style rewriting
3. Marketing and product copy generation

Language Translation

1. Real-time multilingual translation
2. Context-aware translation preserving meaning
3. Cross-border communication support

Coding Assistance & Question Answering

APPLICATIONS — Developer Productivity and Knowledge Retrieval

CODING ASSISTANCE

LLMs significantly improve developer productivity by understanding code context and generating accurate implementations.

NOTABLE TOOLS

GitHub Copilot

AI code autocompletion and contextual coding support

Claude Code

AI-assisted coding with project-level context awareness

CAPABILITIES

1. Code generation from natural language comments
2. Bug detection and step-by-step debugging
3. Refactoring and code quality suggestions
4. Test case and documentation generation

QUESTION ANSWERING

LLMs act as knowledge-based systems trained on large datasets, capable of answering a vast range of questions.

EXAMPLE QUERIES

"Why is the sky blue?"

"Explain quantum computing simply."

"How does async/await work in Python?"

HOW IT WORKS

The model retrieves patterns from its training data and generates structured explanations — functioning like an intelligent knowledge assistant that synthesises rather than looks up.

Unlike search engines, LLMs synthesise and reason over knowledge — not just retrieve it.

Limitations & Reasoning Challenges

LIMITATIONS — Fundamental Constraints of LLMs

01 Complex Reasoning

LOGICAL PROCESSING

Struggles with multi-step logical, mathematical, and symbolic problems — relies on pattern matching rather than true reasoning.

SYMPTOMS

- Multi-variable logic problems
- Long dependency reasoning chains
- Symbolic computation and algebra

EXAMPLE

12 × 15 may produce errors from pattern matching alone

02 Context Window

MEMORY CONSTRAINTS

Limited working memory for long conversations and documents — information beyond the context limit is simply unavailable to the model.

SYMPTOMS

- Earlier information lost beyond capacity
- Reduced accuracy in long-form tasks
- Token limit constraints on input size

EXAMPLE

Models may contradict themselves after long exchanges

03 Hallucination

OUTPUT RELIABILITY

Generates confident but incorrect or fabricated outputs — the model optimises for plausible text rather than factual accuracy.

SYMPTOMS

- Fabricated references and citations
- Inconsistent responses to same query
- Probabilistic generation variance

EXAMPLE

Cited papers and statistics may not exist

Despite these limitations, combining LLMs with reasoning chains, retrieval, and tool use significantly mitigates each constraint.

Mitigation Strategies

LIMITATIONS — Reasoning Methods and Tool Integration

CHAIN-OF-THOUGHT REASONING

Models generate intermediate reasoning steps before the final answer — making the process transparent and improving structured problem solving.

BENEFITS

- ✓ Improves transparency of model reasoning
- ✓ Enhances structured and multi-step problem solving
- ✓ Reduces logical errors in complex tasks

TRADE-OFFS

- • Increases token usage per query
- • Can introduce interruptions in reasoning flow

TOOL INTEGRATION

Connecting LLMs to external tools improves reliability and real-time accuracy beyond static training data.

Calculator
Accurate arithmetic

Search Engines
Up-to-date information

Databases
Real-time data access

Code Execution
Runtime validation

TOOL USAGE WORKFLOW

1. Identify need
2. Call tool
3. Receive results
4. Finalise response

Enables grounded reasoning and reduces dependency on internal knowledge alone.

Challenges & Risks

RISKS — Security, Ethics, and Mitigation

SECURITY RISKS

Prompt Injection
Manipulating instructions to override safeguards

Jailbreaking
Bypassing safety constraints via crafted prompts

Data Poisoning
Injecting harmful data into the training pipeline

Backdoor Attacks
Hidden triggers embedded to alter model behaviour

ETHICAL CONSIDERATIONS

Training Data Bias
Outputs reflect and amplify biases in training data

Misinformation
Hallucinated content spread and accepted as fact

Privacy Risks
Sensitive data processed without adequate controls

Environmental Impact
Large-scale training and inference energy costs

MITIGATION APPROACHES

- ✓ Human oversight and evaluation
- ✓ Robust alignment training
- ✓ Red-team testing for vulnerabilities
- ✓ Continuous model updates
- ✓ Policy controls and usage restrictions

No single solution solves all risks. Modern systems combine technical safeguards with governance mechanisms.

Multimodal AI

EMERGING AI — Processing Text, Images, Audio, and Video Together

WHAT IT PROCESSES

Multimodal AI combines multiple data types into a single model capable of reasoning across all of them simultaneously — unlike earlier systems limited to one modality.

- **Text**
- **Images**
- **Audio**
- **Video**

SHAPING THE FUTURE

1. 50% of consumers now prefer multimodal interactions as their primary format
2. Agents can analyse videos, PDFs, code, and speech in a single session
3. Healthcare, retail, and education are being transformed by unified perception models

LEADING PLAYERS

The four models leading the multimodal frontier in 2026:

Google Gemini 3.1 Pro

Text, image, audio, video + 1M token context

OpenAI GPT-5

Native voice, vision & real-time interaction

Anthropic Claude Opus 4.6

Document, image & code understanding

Meta Llama 4

Open-weight multimodal, 10M token context

Autonomous AI Agents

EMERGING AI — Planning, Executing, and Iterating with Minimal Human Input

40% Enterprise apps with AI agents by end 2026

$52B Projected agentic AI market by 2030

90% Cost reduction with Plan-and-Execute agents

HOW THEY WORK

1. **Perceive**
   Process multimodal inputs from the environment
2. **Plan**
   Break high-level goals into executable sub-tasks
3. **Execute**
   Call APIs, write and run code, interact with tools
4. **Reflect**
   Evaluate outcomes and self-correct on failure
5. **Orchestrate**
   Delegate work to specialised sub-agents

KEY PLATFORMS

OpenAI Operator
Web-native task automation

Claude Code
Agentic software development

Cursor / Windsurf
AI coding with sub-agent capabilities

LangGraph / AutoGen
Multi-agent orchestration frameworks

n8n / Gumloop
No-code agent builders for workflows

By 2027, 70% of multi-agent systems will use narrow, specialised agents rather than generalist ones — improving accuracy significantly.

Voice-Based AI Systems

EMERGING AI — Natural, Bidirectional Voice AI with Emotional Intelligence

THE EVOLUTION OF VOICE AI

2010s — Command-Response

Siri, Alexa — scripted, turn-based, limited vocabulary

↓

2022–24 — LLM-Powered

Contextual understanding and memory, but text-first interfaces

↓

2025 — Real-Time Voice

GPT-4o, Gemini Live — fully conversational, low-latency responses

↓

2026 — Agentic Voice

Emotionally intelligent, action-capable, workflow-aware voice agents

2026 CAPABILITIES

- **Sub-200ms latency**
- **Emotion detection**
- **24 languages**
- **Voice workflows**

LEADING PLATFORMS

OpenAI GPT-4o Voice
Realtime API with emotion detection

Google Gemini Live
Multimodal voice + video + screen sharing

ElevenLabs
Ultra-realistic voice cloning & synthesis

Hume AI
Empathic voice AI with emotional intelligence

Bland AI / Vapi
Enterprise voice agent infrastructure

Emotional AI market: $19.5B (2020) → $37.1B (2026)

Physical AI & Embodied Intelligence

EMERGING AI — AI Systems that Perceive, Reason, and Act in the Physical World

Humanoid Robots

Figure 02, Tesla Optimus, Unitree G1 & H2 — general-purpose robots learning from video demonstrations and LLM-based planning. Moving toward household and industrial task execution.

Autonomous Vehicles

Waymo, Tesla FSD, Zoox — navigating unstructured environments using real-time multimodal sensor fusion. Waymo operates fully driverless commercial fleets in multiple US cities.

Industrial Robotics

Boston Dynamics Spot, Amazon Proteus — warehouse, logistics, and factory automation with adaptive manipulation. AI handles variability that previously required hard-coded programming.

Drone Swarms & UAVs

Zipline, Shield AI — autonomous aerial agents for medical delivery, surveillance, and emergency response. Shield AI's Hivemind enables coordinated multi-drone missions.

AI is escaping the screen — the physical world is becoming the next frontier for autonomous intelligence.

Frontier AI Models — 2026

AI LANDSCAPE 2026 — No Single Model Dominates: Route by Strength

Gemini 3.1 Pro

Google DeepMind

BEST FOR

Abstract reasoning, multimodal

- #1 ARC-AGI-2 (77.1%)
- 1M token context window
- $2 / $12 per 1M tokens

Claude Opus 4.6

Anthropic

BEST FOR

Agentic workflows, code review

- #1 SWE-Bench Verified (80.8%)
- #1 Agentic Elo (1,606)
- HLE with tools: 53.1%

GPT-5.2

OpenAI

BEST FOR

Software engineering, enterprise

- Leader on SWE-Bench Pro
- GPQA Diamond: 92.4%
- Strong multimodal support

Qwen 3.5

Alibaba

BEST FOR

Open-source, multilingual

- 397B MoE architecture
- Open-weight, self-hostable
- Leading multilingual scores

Key Benchmarks & Performance

AI LANDSCAPE 2026 — Source: February–March 2026 Data

Benchmark	What It Tests	Gemini 3.1 Pro	Claude Opus 4.6	GPT-5.2 Pro
ARC-AGI-2	Novel abstract reasoning	**77.1%**	68.8%	54.2%
HLE (no tools)	Expert-level knowledge	**44.4%**	40.0%	36.6%
HLE (with tools)	Tool-augmented reasoning	51.4%	**53.0%**	50.0%
GPQA Diamond	Graduate-level science	**94.3%**	91.3%	93.2%
SWE-Bench Verified	Real-world code fixes	80.6%	**80.8%**	80.0%
SWE-Bench Pro	Complex software engineering	54.2%	—	**55.6%**
Agentic Elo	Expert-level office work	1,317	**1,606**	1,462

2026 marks the end of one-model dominance. Route by task: Gemini for abstract reasoning, Claude for agentic work, GPT-5.2 for engineering depth.

Companies Leading the AI Space

AI LANDSCAPE 2026 — Frontier Labs, Open-Weight, and Agent-Native Startups

Tier 1 — Frontier Model Labs	Tier 2 — Open-Weight	Tier 3 — Agent-Native
Anthropic Claude 4 family; leads agentic workflows & safety	**Meta AI (Llama 4)** 10M token context; fuels open-source ecosystem	**Cursor / Windsurf** AI-first coding IDEs with multi-agent execution
OpenAI GPT-5 series; dominant enterprise adoption	**Alibaba (Qwen 3.5)** 397B MoE; leading open-weight multilingual	**ElevenLabs** Leading voice synthesis & cloning platform
Google DeepMind Gemini 3.x; benchmark crown; unmatched multimodal	**DeepSeek** Frontier-class at $0.28 / 1M tokens	**Vellum / Gumloop / n8n** Agentic workflow automation platforms
xAI (Grok) Grok 4.x; parallel multi-agent architecture	**Mistral AI** Efficient MoE; strong EU open-source presence	**Hume AI** Empathic voice AI; emotion-aware systems

The AI stack has stratified: frontier labs own the base models, open-weight democratises access, and agent-native startups own the application layer.